

TOO. Historique

- Naissance du paradigme OO en 1967 avec Simula
- But essentiel : Réduire et gérer la complexité des logiciels
 - Décomposition modulaire
 - Regroupement des fonctions et des propriétés concernant un type donné dans un module ou une entité
 - Cacher la complexité des fonctions et celle de leurs actions
 - Fournir une interface qui sera la partie visible du module
 - Communication par envoi de messages
- Développement de SmallTalk (70) et de ADA
- Sophistication des interfaces utilisateur graphiques (80)
- Apparition des langages hybrides (Java, C++, etc.)
- BD OO, Systèmes distribués, Méthodes OO (90)

TOO. Historique

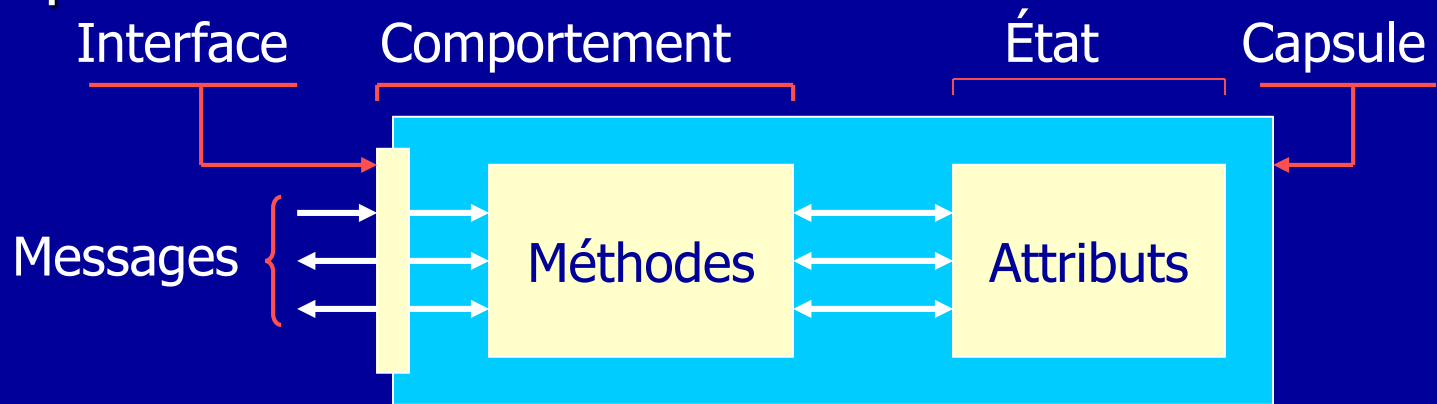
- Nécessité d'une méthode d'analyse et de conception OO
 - Au début, les chevronnés concevaient des systèmes OO de façon intuitive. Les autres éprouvaient de la difficulté
- L'ère des MACOO (Méthode d'Analyse et de Conception OO)
 - OOA de Coad & Yourdon
 - Méthode de Shlaer et Mellor
 - OOM (Merise orienté objet)
 - OMT (Rumbaugh et al. 1991)
 - Objectory (Jaconbson et al. 1992)
 - Méthode de Booch (1994)
- MACOO hybrides (Exemple : HOOM pour Hybrid OO Method)
- Vers une standardisation
 - UML (Langage de modélisation unifié)
 - Donne une notation standard pour le développement OO

TOO. Objet

- Un objet représente une entité physique, conceptuelle ou logicielle
- Un objet est une entité unique, distincte et identifiable
- Exemples
 - Une table, un homme, un logiciel, une administration, etc.
- Un objet possède
 - **Un état** : Implanté par des propriétés (attributs) et leurs valeurs
 - **Un comportement** : La façon dont il interagit avec les autres objets (méthodes)
 - **Identité** : Pour le distinguer des autres objets

TOO. Objet

- Récapitulation



- Exemples d'objets

Identité →

RECTANGLE10

Étudiant100

Attributs {

Hauteur : 200
Longueur : 500

Nom : Keïta
Prénom : Ali

Services {

Tracer-Rect()
Effacer()

Parler()
Marcher()

TOO. Classe

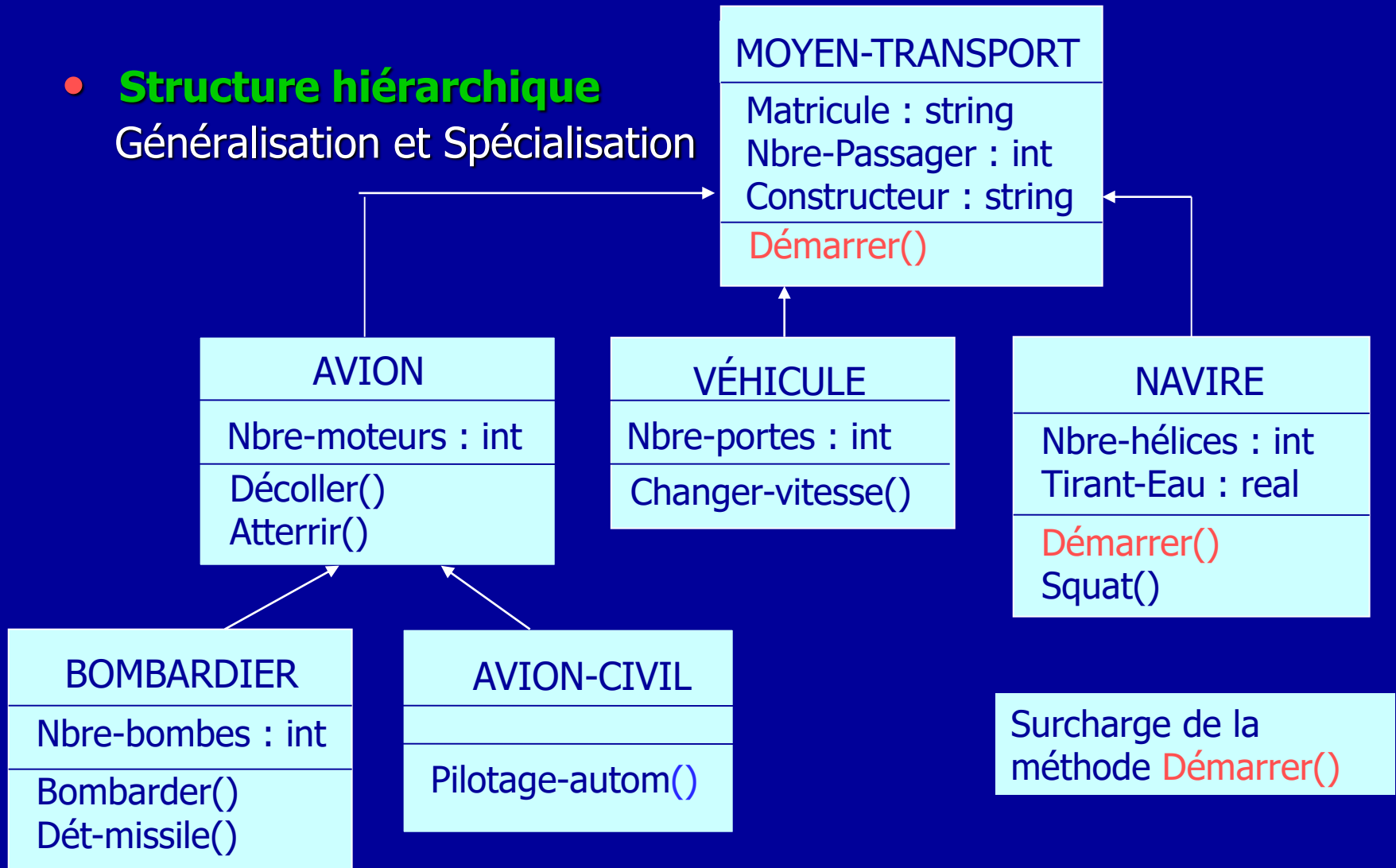
- Une classe est un moule à partir duquel on peut générer un ensemble d'objets partageant des attributs et des méthodes communes
- Une classe est un modèle à partir duquel on peut générer de nouvelles classes ou de nouveaux objets
- Un objet est une instance d'une classe
- Représentation

GRAPHE
Couleur : string
Agrandir() Effacer()

PERSONNE
Nom : string Prénom : string
Parler() Marcher()

TOO. Classe

- **Structure hiérarchique**
Généralisation et Spécialisation



TOO. Classe

- Combien y a-t-il de classes ?

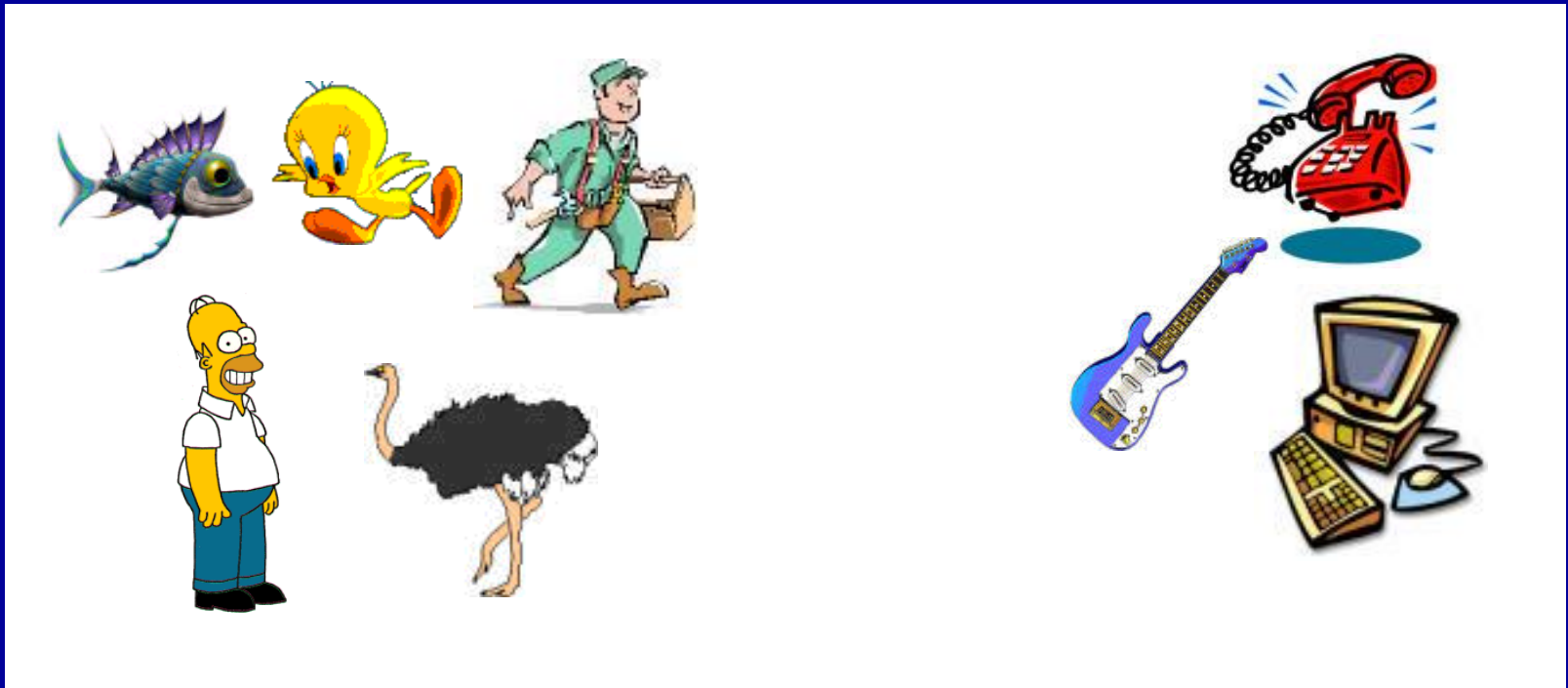


TOO. Classe

- Exemples de classifications possibles
 - Possibilité 1

Être vivant

Matériel



TOO. Classe

- Exemples de classifications possibles
 - Possibilité 2

Personne



Animal



Matériel



TOO. Classe

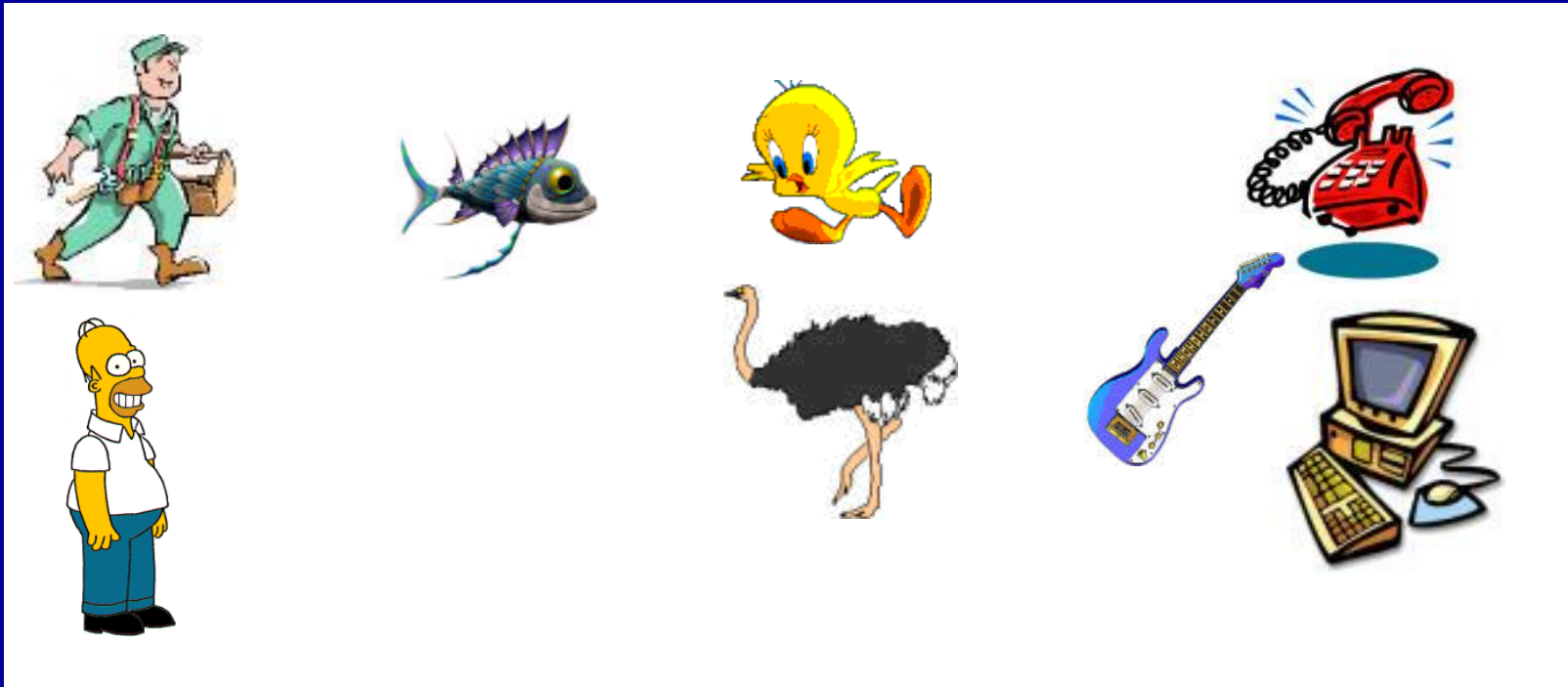
- Exemples de classifications possibles
 - Possibilité 3

Personne

Poisson

Oiseau

Matériel



TOO. Classe

- Exemples de classifications possibles
 - Possibilité 4

Personne	Poisson	Oiseau Volant	Matériel de communication	
				
		Oiseau non Volant	Instrument de musique	
				

- Point important : La détermination des classes dépend du contexte

TOO. Caractéristiques

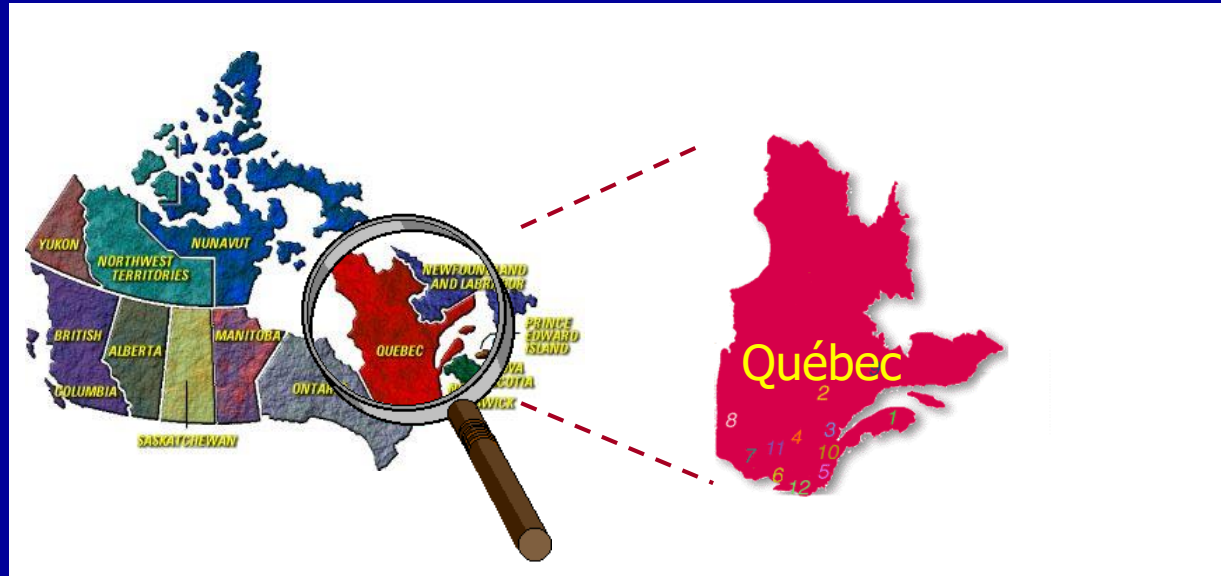
- Abstraction
- Encapsulation
- Héritage
- Communication par envoi de messages
- Polymorphisme
- Agrégation

TOO. Caractéristiques

- Abstraction
 - Un problème n'existe pas tout seul : il fait partie d'un système
 - Comment isoler le problème du reste du système ?
 - Définition : **L'abstraction est l'habileté mentale de voir le domaine du problème avec différents degrés de détails en fonction du contexte courant du problème**
 - Ignorer les aspects qui ne sont pas pertinents pour le contexte et se concentrer sur ce qui est important

TOO. Caractéristiques

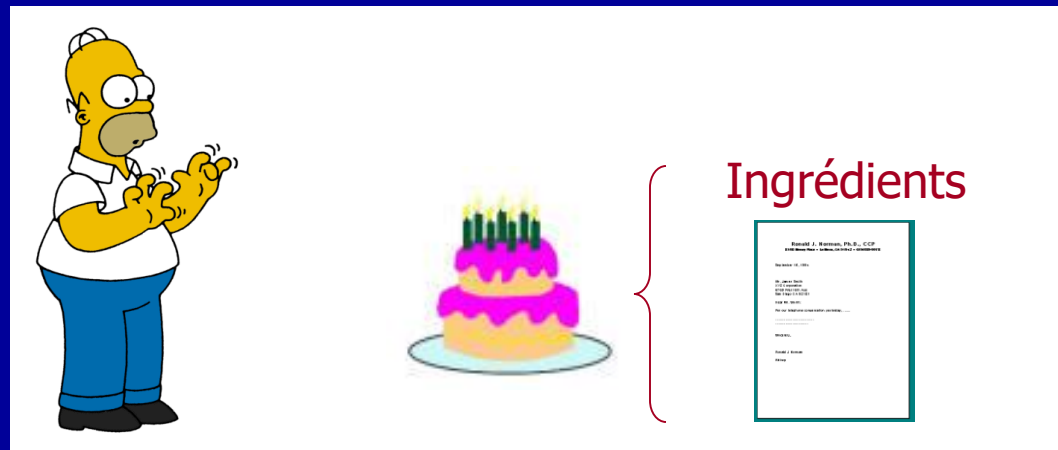
- Abstraction
 - Idée générale



- Approche de résolution d'un problème
 - Identifier le problème
 - Faire abstraction du problème en des objets : **Recherche des objets**
 - Faire des modèles à bases des objets identifiés en identifiant les processus et les données pour ces modèles
 - Construire le système

TOO. Caractéristiques

- Encapsulation
 - Cache de l'information
 - Technique qui isole l'aspect externe de l'objet de son aspect interne
 - Les données et les méthodes sont rassemblées ensemble. Elles sont cachées et protégées
 - Division de l'objet en une interface et un corps
 - Exemple



TOO. Caractéristiques

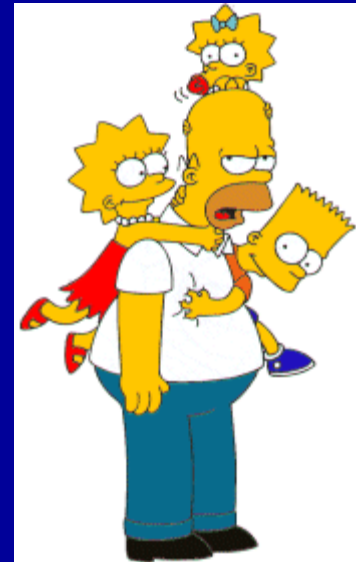
- Héritage
 - Mécanisme de dérivation d'une nouvelle classe ou d'un nouvel objet à partir d'une classe existante
 - L'idée provient de l'existence d'une hiérarchie de classes
 - Un objet hérite les attributs et les méthodes de la classe dont il est issu
 - Une sous-classe peut ajouter des attributs et/ou des méthodes
 - Une sous-classe peut redéfinir une méthode par surcharge

Héritage au niveau de :

Comportement

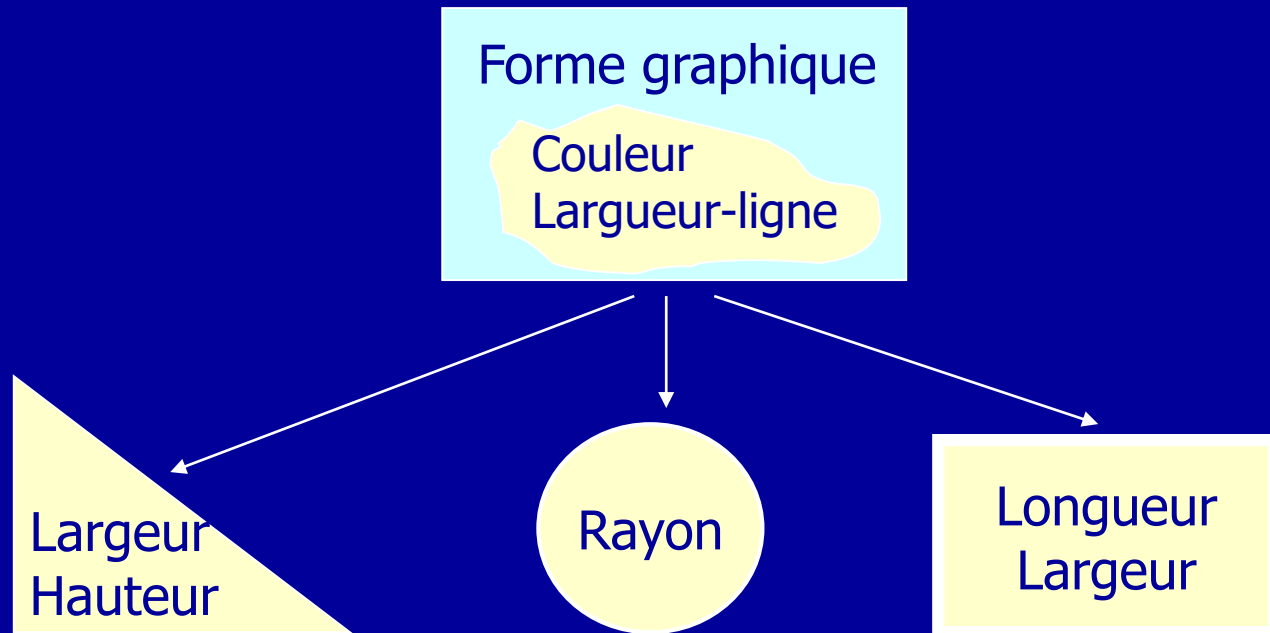
Apparence

Attitude



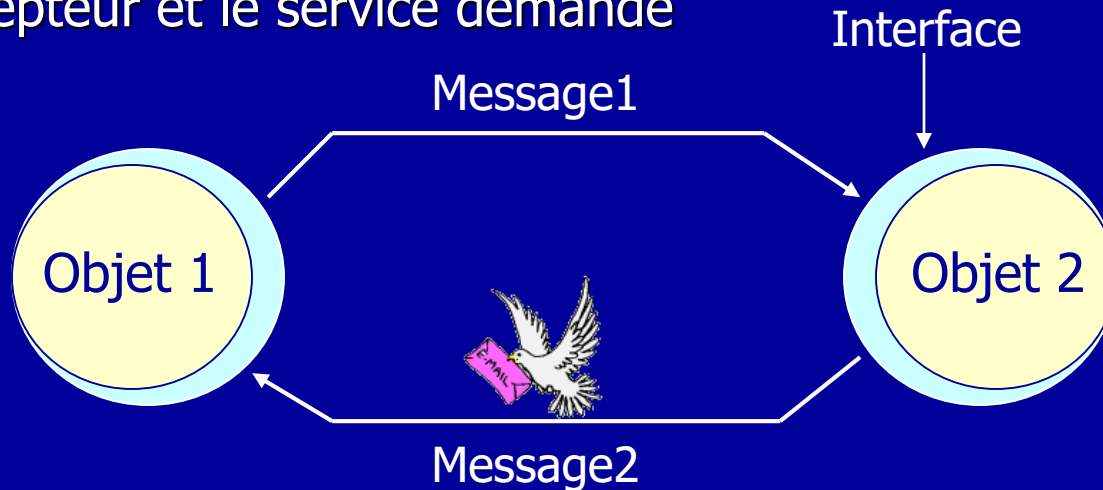
TOO. Caractéristiques

- Héritage
 - On parle de hiérarchie de classes : Super-classe et sous-classe
 - Exemple



TOO. Caractéristiques

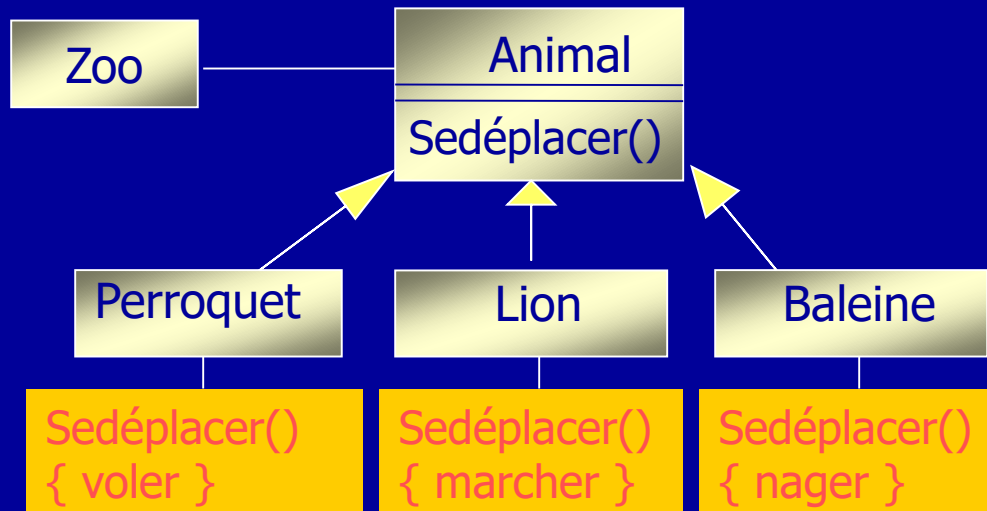
- Communication par envoi de messages
 - Un objet interagit avec les autres objets par envoi de messages
 - Le message contient l'identité de l'expéditeur, l'identité du récepteur et le service demandé



- À travers l'interaction entre les objets, on peut :
 - Réaliser des fonctionnalités de haut niveau
 - Créer des comportements complexes

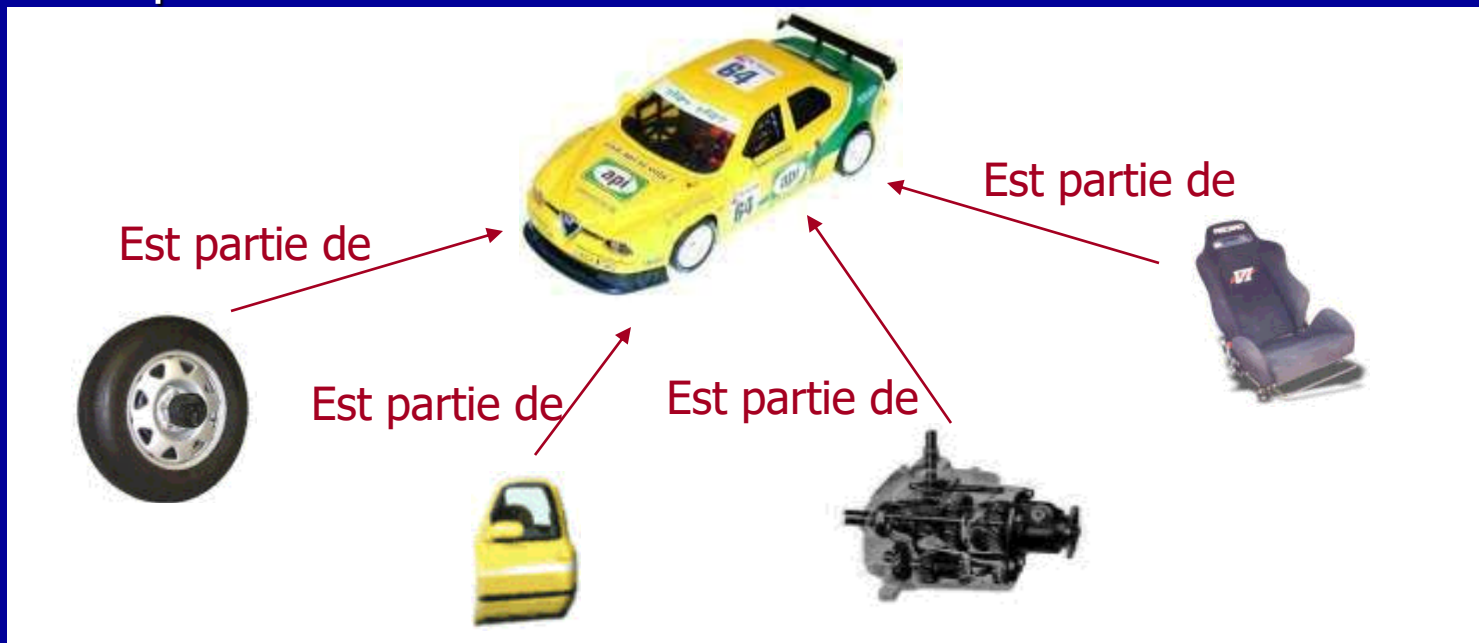
TOO. Caractéristiques

- Polymorphisme (plusieurs formes)
 - C'est l'habileté de cacher différentes implantations derrière une interface commune
 - Décrit la caractéristique d'un élément qui peut prendre plusieurs formes (Ex. L'eau peut être liquide, glace et vapeur)
 - (en info.) Fait référence à un objet qui peut désigner des instances de classes différentes issues d'une même arborescence



TOO. Caractéristiques

- Agrégation
 - Relation entre deux classes spécifiant que des objets d'une classe sont les **composants** de l'autre classe
 - L'agrégation spécifie la relation «**est partie de**»
 - Exemple



TOO. En résumé ...

- Une approche orientée objet c'est :
 - Une recette qui contrôle la démarche vers l'obtention du système souhaité
 - Une solution destinée à faciliter l'évolution d'applications complexes
 - Un ensemble de concepts stables, éprouvés et normalisés
 - Une panoplie d'outils et de langages performants pour le développement

TOO. Avantages

- Une meilleure description et compréhension du monde réel modélisé par des objets
- Encapsulation
 - Un changement des besoins n'affecte pas tout le système
 - Meilleure gestion de la complexité (cachette de l'information)
 - Lisibilité
 - Interface = Mode d'emploi
- Héritage
 - Permet la réutilisation des objets/classes créés (gain en rapidité)
 - Suppression des redondances

TOO. Avantages

- Qualité supérieure des programmes créés à partir des composants existants testés
- Abstraction
 - L'interface permet de définir un objet sans s'inquiéter des détails de l'implantation ➡ Se concentrer sur l'important
 - Possibilité d'utilisation du modèle objet de l'analyse au développement
- Maintenance simplifiée : Changement effectué à une seule place

TOO. Avantages

- Extensibilité
 - Des systèmes complexes sont créés à partir de sous-systèmes développés et testés indépendamment (réutilisation)
 - Changement du système existant sans la reconstruction d'un nouveau système

TOO. Inconvénients

- Détermination des classes et des objets dépend des auteurs (abstraction et contexte)
- Réutilisation prudente des composants
- Approche moins intuitive
 - Demande un niveau d'abstraction élevé de la part du concepteur/analyste
 - Décomposition du problème en une hiérarchie de fonctions et de données et en terme d'interaction entre les objets

Références ...

- J. Lam, *Object-Oriented Technologie*
- P. Papajorgji, *Introduction to object Oriented Design and UML*
- R. Norman, *Object-Oriented System Analysis Review*
- P.A Muller et N. Gaertner. *Modélisation objet avec UML*
- <http://uml.free.fr/cours/p4.html>